DTypeConfig#

https://pytorch.org/docs/stable/generated/torch.ao.quantization.backend_conf

Description:

Config object that specifies the supported data types passed as arguments to quantize ops in the reference model spec, for input and output activations, weights, and biases. For example, consider the following reference model: quant1 - [dequant1 - fp32_linear - quant2] - dequant2 The pattern in the square brackets refers to the reference pattern of statically quantized linear. Setting the input dtype as torch.quint8 in the DTypeConfig means we pass in torch.quint8 as the dtype argument to the first quantize op (quant1). Similarly, setting the output dtype as torch.quint8 means we pass in torch.quint8 as the dtype argument to the second quantize op (quant2). Note that the dtype here does not refer to the interface dtypes of the op. For example, the “input dtype” here is not the dtype of the input tensor passed to the quantized linear op. Though it can still be the same as the interface dtype, this is not always the case, e.g. the interface dtype is fp32 in dynamic quantization but the “input dtype” specified in the DTypeConfig would still be quint8. The semantics of dtypes here are the same as the semantics of the dtypes specified in the observers.

Function Signature

```
class
torch.ao.quantization.backend_config.DTypeConfig(input_dtype
=None, output_dtype=None, weight_dtype=None,
bias_dtype=None, is_dynamic=None)[source]#

classmethod from_dict(dtype_config_dict)[source]#

Create a DTypeConfig from a dictionary with the following
items (all optional):

Return type

to_dict()[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> dtype_config1 = DTypeConfig(
...     input_dtype=torch.quint8,
...     output_dtype=torch.quint8,
...     weight_dtype=torch.qint8,
...     bias_dtype=torch.float)

>>> dtype_config2 = DTypeConfig(
...     input_dtype=DTypeWithConstraints(
...         dtype=torch.quint8,
...         quant_min_lower_bound=0,
...         quant_max_upper_bound=255,
...     ),
...     output_dtype=DTypeWithConstraints(
...         dtype=torch.quint8,
...         quant_min_lower_bound=0,
...         quant_max_upper_bound=255,
...     ),
...     weight_dtype=DTypeWithConstraints(
...         dtype=torch.qint8,
...         quant_min_lower_bound=-128,
...         quant_max_upper_bound=127,
...     ),
...     bias_dtype=torch.float)

>>> dtype_config1.input_dtype
torch.quint8

>>> dtype_config2.input_dtype
torch.quint8

>>> dtype_config2.input_dtype_with_constraints
DTypeWithConstraints(dtype=torch.quint8,
quant_min_lower_bound=0, quant_max_upper_bound=255,
scale_min_lower_bound=None, scale_max_upper_bound=None)
```

Detailed Documentation

Documentation

Config object that specifies the supported data types passed as arguments to quantize ops in the reference model spec, for input and output activations, weights, and biases.

For example, consider the following reference model:

quant1 - [dequant1 - fp32_linear - quant2] - dequant2

The pattern in the square brackets refers to the reference pattern of statically quantized linear. Setting the input dtype as torch.quint8 in the DTypeConfig means we pass in torch.quint8 as the dtype argument to the first quantize op (quant1). Similarly, setting the output dtype as torch.quint8 means we pass in torch.quint8 as the dtype argument to the second quantize op (quant2).

Note that the dtype here does not refer to the interface dtypes of the op. For example, the “input dtype” here is not the dtype of the input tensor passed to the quantized linear op. Though it can still be the same as the interface dtype, this is not always the case, e.g. the interface dtype is fp32 in dynamic quantization but the “input dtype” specified in the DTypeConfig would still be quint8. The semantics of dtypes here are the same as the semantics of the dtypes specified in the observers.

These dtypes are matched against the ones specified in the user’s QConfig. If there is a match, and the QConfig satisfies the constraints specified in the DTypeConfig (if any), then we will quantize the given pattern using this DTypeConfig. Otherwise, the QConfig is ignored and the pattern will not be quantized.

“input_dtype”: torch.dtype or DTypeWithConstraints “output_dtype”: torch.dtype or

DTypeWithConstraints “weight_dtype”: torch.dtype or DTypeWithConstraints “bias_type”: torch.dtype “is_dynamic”: bool

Convert this DTypeConfig to a dictionary with the items described in from_dict().
